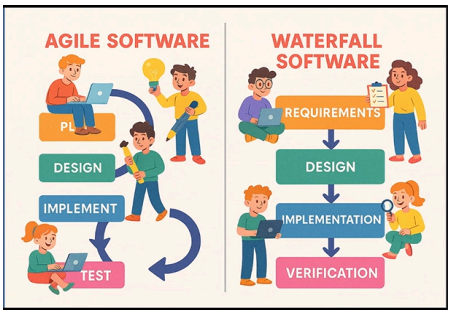
Ingeniería y Calidad de Software - parcial 2

Testing en Ambientes Ágiles - clase 09/10



Enfoque tradicional / Waterfall software:
Encontrar defectos. Se testea luego de que el requerimiento se implementó
Enfoque Agile:
Construir a partir del testing. Flujo que se alimenta de manera continua, las pruebas se van ejecutando en la implementación también. Solo se implementa el testing ágil si estoy usando una metodología ágil.

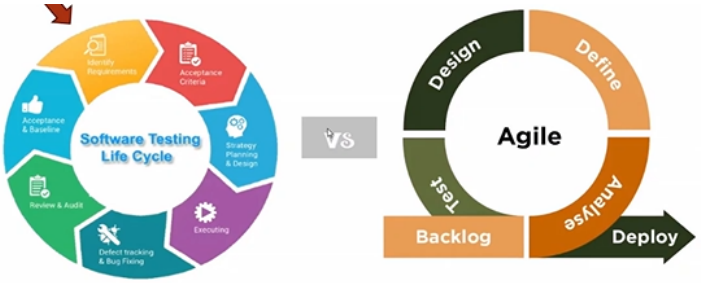
Manifiesto del Testing:
valores:

- pruebas durante **sobre** pruebas al final
- prevenir defectos **sobre** encontrar defectos
- entender lo que se está probando **sobre** verificar la funcionalidad
- construir un mejor sistema **sobre** romper el sistema
- responsabilidad de todo el equipo por la calidad **sobre** el tester es responsable de la calidad

El testing no es una actividad, es una actividad parte de la implementación.
El testing no es encontrar defectos, es prevenirlos
El testing no es testear técnicamente, si no entender el dominio
El testing no es romper el sistema, si no crear el mejor sistema
Todo el equipo es responsable de la calidad

Ciclo de vida Agile vs Software Testing
Software testing:
conformidad con los requerimientos

Agile:
entrega de valor continuo, ocurre a lo largo de todas las actividades



Principios del testing

1. Testing se mueve hacia adelante en el proyecto (retroalimentación continua)
2. Testing no es una fase
3. Todos hacen testing
4. Reducir la latencia del feedback
5. Las pruebas representan expectativas
6. Mantener el código limpio, corregir los defectos rápido
7. Reducir la sobrecarga de documentación de las pruebas
8. Las pruebas son parte del “done”
9. De probar al final a Conducido por Pruebas

Prácticas Concretas → lo que finalmente terminó implementando

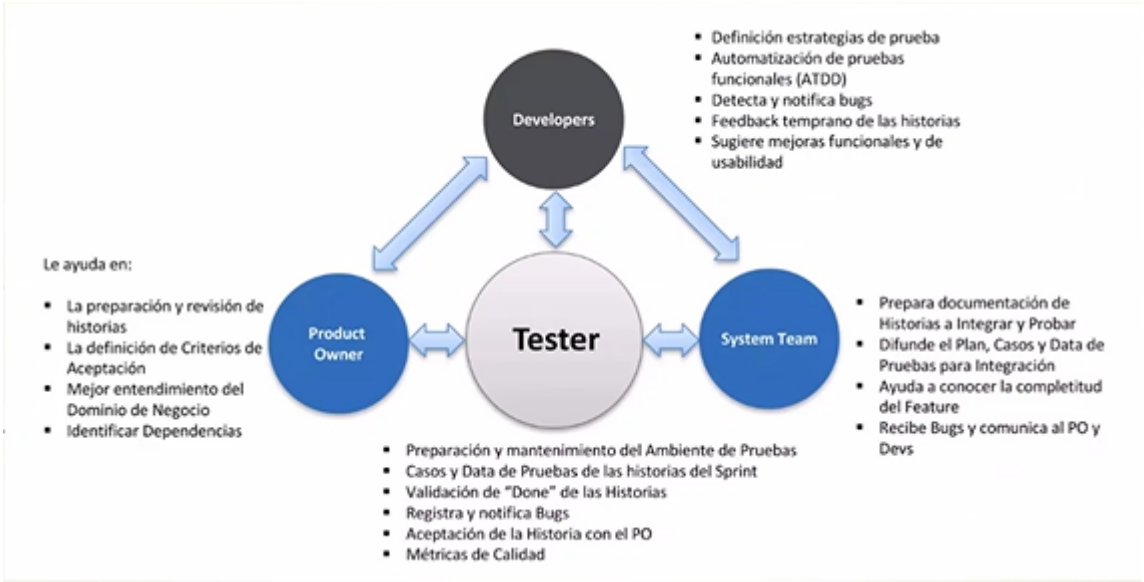
1. Pruebas de unidad e integración automatizadas
2. Pruebas de Regresión a nivel de sistema automatizadas → volver a probar las cosas que ya funcionaban luego de solucionar un bug
3. Pruebas exploratorias
4. TDD (Test Driven Development)
5. ATDD (Desarrollo conducido por Pruebas de aceptación)
6. Control de versión de las pruebas con el código

Desarrollo conducido por pruebas de aceptación (ATDD)

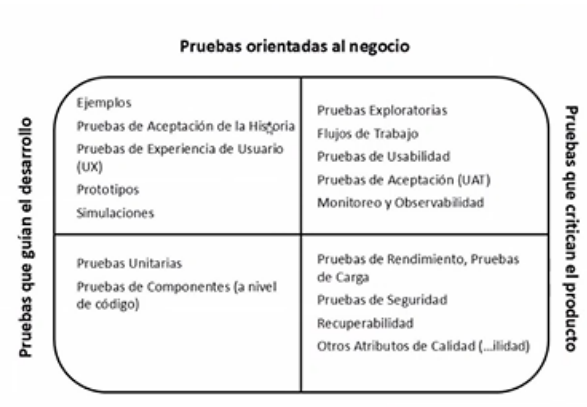
1. pruebas de aceptación de alto nivel
2. automatización de las pruebas
3. programar la solución

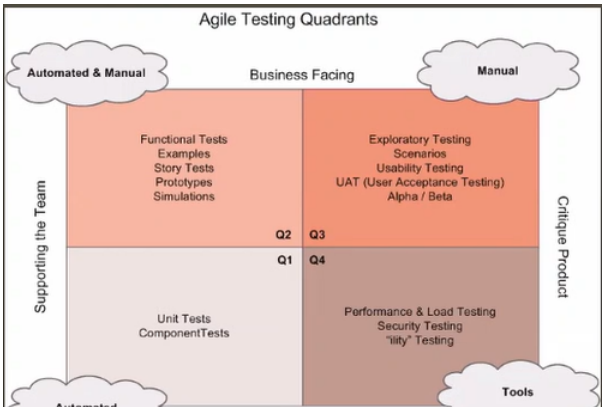


El rol del Tester en equipos ágiles:



Cuadrantes del Testing Ágil

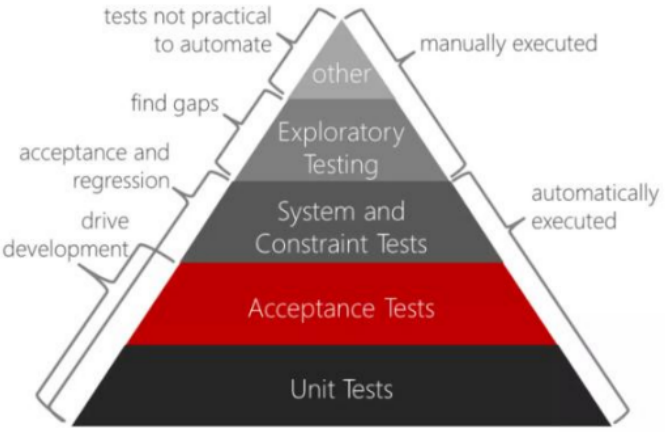




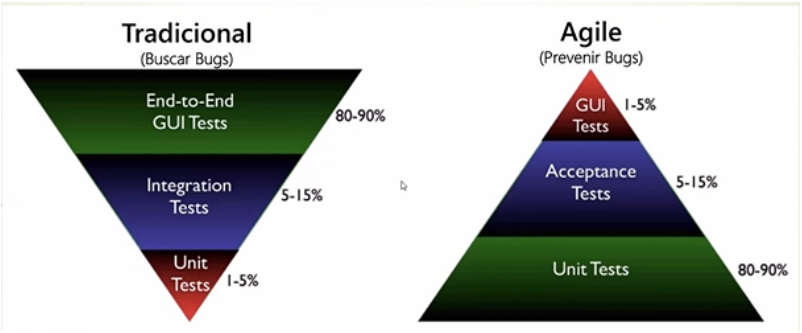
Q1: pruebas orientadas a la tecnología, pruebas que guían al desarrollo → apunta a la calidad interna del código fuente y a la arquitectura del software
Q2: pruebas que guían al desarrollo, pruebas orientadas al negocio → validar que las funcionalidad desarrolladas cumplan con las especificaciones de negocio
Q3: pruebas orientadas al negocio, pruebas que critican al producto → producto evaluado a partir del usuario final
Q4: pruebas que critican al producto, pruebas orientadas a la tecnología → verificación de los atributos no funcionales

Pirámide del Testing Agile

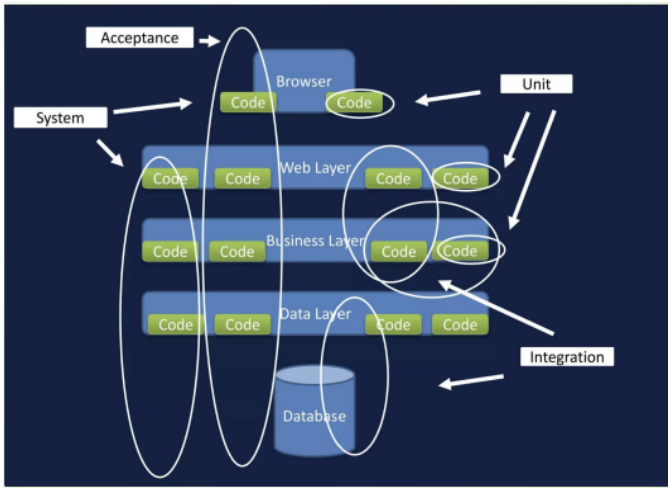




Pirámide del Testing Tradicional vs Agile



Niveles de prueba



Beneficios de la automatización del testing

- rápida ejecución
- reusabilidad
- mayor cobertura
- mayor precisión
- mayor alcance
- reducción de costos

Lean Kanban - clase 16/10

Filosofia Lean

Analizar el proceso que queremos mejorar e identificar qué tipos de desperdicio tenemos, y qué hacer con ellos.

Principios:

→ Amplificar Aprendizaje
→ Embeber la Integridad conceptual
→ Diferir compromisos
→ Dar poder al equipo
→ Ver el todo
→ Entregar lo antes posible
→ **Eliminar Desperdicios → mejora de procesos**

Desperdicio/Gastos en producción Lean:

- producción en exceso
- stock
- pasos extra en el proceso
- busqueda de informacion
- defectos
- esperas
- transportes
- **talento no utilizado**

Tipos de desperdicio:

→ Tipo 1: desperdicio necesario → minimizar
los que no se pueden eliminar del proceso
ej: tests
→ Tipo 2: puro desperdicio → evitar
es necesario eliminarlos

Comparación con desperdicios en servicios basados en conocimiento:

Desperdicios en Manufactura	Desperdicios en servicios basados en conocimiento
Inventario	Trabajo parcialmente terminado
Sobreproducción	Características extra
Proceso extra	Proceso extra
Transporte	Cambio de tareas Transferencia de conocimiento
Esperas	Esperas/Demoras
Movimiento	Cambio de contextos (Seteo)
Defectos	Defectos
Talento no utilizado	Talento no utilizado

Kanban

Es un método para definir, gestionar y mejorar servicios que entregan trabajo del conocimiento, tales como servicios profesionales, trabajos o actividades en las que interviene la creatividad y el diseño tanto de productos de software como físicos.

kan-ban = signal-card

Valores de Kanban:

- Transparencia
- Equilibrio
- Colaboración
- Foco en el cliente
- **Flujo**
- Liderazgo
- Entendimiento
- Acuerdo
- Respeto

Principios de Kanban:

Gestión de Cambios	Comenzar con lo que haces ahora: - Entender los procesos actuales tal y como están siendo realizados en la actualidad - Respetar los roles actuales, las responsabilidades de cada persona y los puestos de trabajo
	Acordar la búsqueda de la mejora a través del cambio evolutivo
	Fomentar actos de liderazgo a todos los niveles
Entrega de Servicios	Comprender y enfocarse en cumplir las necesidades y expectativas del cliente
	Gestionar el trabajo; dejar que los trabajadores se auto organicen en torno a él
	Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados

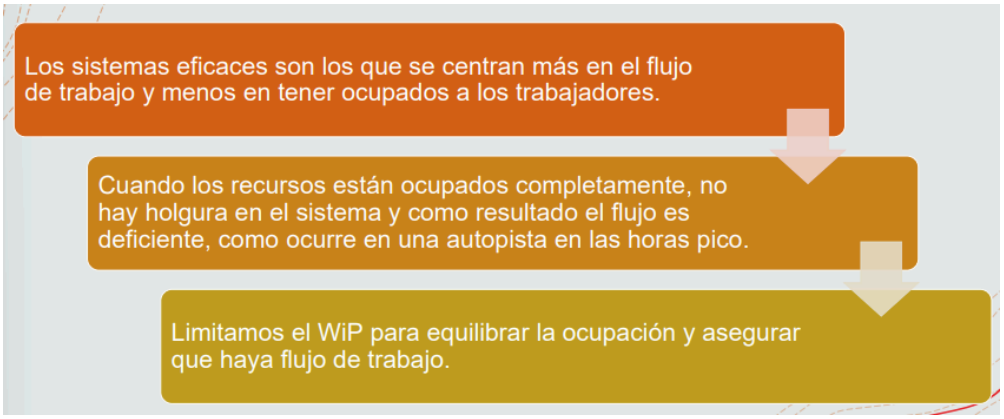
Prácticas generales de Kanban:

→ **Visualizar:** tablero. Muestra el trabajo y su flujo. Visualiza los riesgos. Construye un modelo visual que refleje cómo se trabaja.

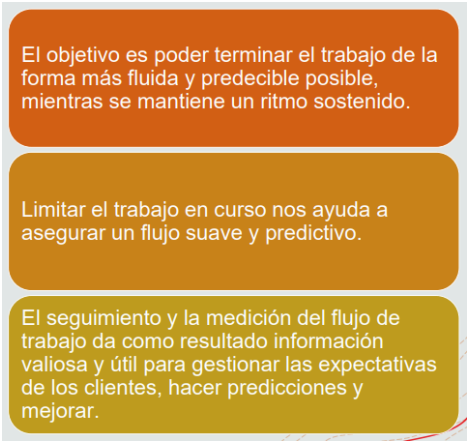
- permite absorber y procesar una gran cantidad de información en un corto intervalo de tiempo
- habilita la cooperación, ya que todo el mundo tiene la misma imagen
- permite tomar decisiones de una manera mas rapida y colaboración

→ **Limitar el trabajo en curso:** deja de empezar, empieza a terminar. De derecha a izquierda. Limita el trabajo en el sistema a la capacidad disponible, basándose en los datos.

- provoca conversación y mejora
- límite WIP → asignar límites explícitos de cuántos ítems puede haber en progreso en cada estado del flujo de trabajo



→ **Gestionar el flujo:** el flujo es el movimiento de trabajo. Gestiona el flujo para ser predecible y confiable. Utiliza los datos.



- administración de colas: foco es en flujo “fin a fin”, flow = centrado en el cliente.
- pull, no push
- modelar el proceso

Cola de Producto	Análisis		Desarrollo		Listo para Build	En Testing		En Producción
	En progreso	Hecho	En progreso	Hecho		En Progreso	Listo para Despliegue	

- dividir el trabajo en piezas
- definir tipos de trabajo: requerimientos, defectos, desarrollo y solicitudes.

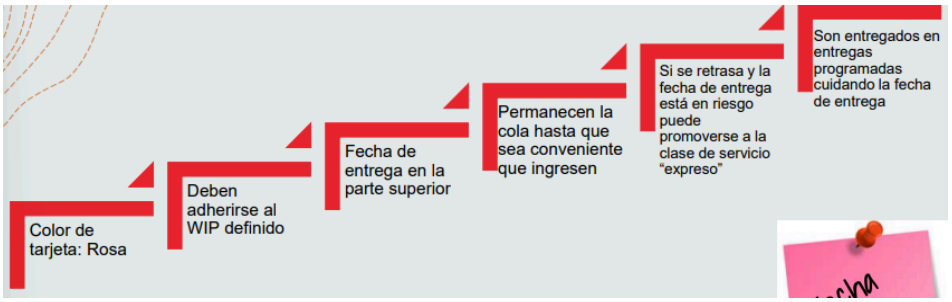
→ **Hacer las políticas explícitas.** Tener políticas acordadas, visibles para todos los involucrados: criterios de pull (atracción/arrastre), límites de WIP (trabajo en curso), clases de servicio, otros que correspondan.

- todas las políticas deben ser acordadas entre todas las partes involucradas, incluyendo a los clientes, interesados y trabajadores responsables del trabajo que está en el tablero.
- las políticas deben estar expuestas en un lugar destacado.
- a nivel de equipo, los acuerdos son una buena forma de introducir las políticas.

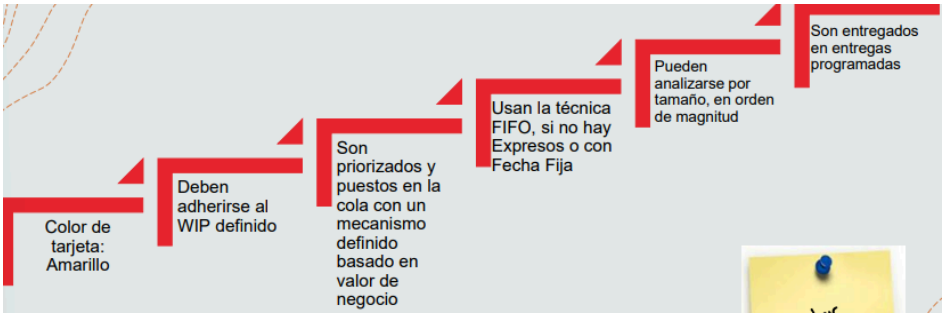
Las políticas deben ser: pocas, sencillas, bien definidas, visibles, aplicables en todo momento y fácilmente modificables por los que prestan el servicio.
Políticas Explícitas para cada **clase de servicio**:
- Política para la clase de servicio “Expreso”:



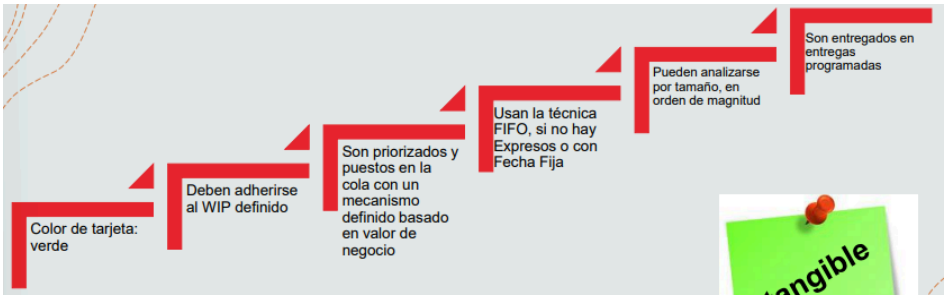
- Política para la clase de servicio “Fecha Fija”:



- Política para la clase de servicio “Estándar”:



- Política para la clase de servicio “Intangible”:



Definir **políticas de calidad** sobre:

- reposición de trabajo en el tablero (cuando, cuanta, cantidad, por quien…)
- definición de cuándo una actividad está terminada y ese elemento de trabajo puede continuar a través del flujo (criterios de tracción)
- límites del trabajo en curso (WiP limits)
- políticas para gestionar elementos de trabajo de diferentes clases de servicio
- cadencia de las reuniones (horarios y agendas)
- otros principios y acuerdos de colaboración

→ **Establecer ciclos de retroalimentación:** con la cadencia adecuada. Fomenta la colaboración, el aprendizaje y las mejoras. Basado en datos.

→ **Mejora colaborativamente, evolucionar experimentalmente: usando el método científico.** El cambio impulsado por hipótesis. Ejecutar experimentos para avanzar o aprender (safe-to-fail experiments).

- “comenzamos con lo que hacemos ahora” y “acordamos la búsqueda de la mejora a través del cambio evolutivo”
- cambio colaborativo
- utilizando experimentos diseñados basados en modelos y en el modelo cientifico

- diseñamos experimentos en entornos donde fallar es seguro de tal forma que si nuestra hipótesis es correcta y el experimento da buenos resultados, mantenemos el cambio
- si el resultado no es positivo, podemos fácilmente volver al estado anterior

¿Cómo aplicar Kanban?
- Proceso: modelar nuestro proceso
- Trabajo: decidir la unidad de trabajo
- Límites de WIP: limitar el WIP para ayudar al flujo de trabajo
- Política: definir políticas de calidad
- Cuellos de botella y flujo: mover recursos a los cuellos de botella
- Clase de servicio: diferentes trabajos tienen diferentes políticas - definición de hecho ("done"), para cada estado
- Cadencia: releases, planificaciones, revisiones

Cadencias/Eventos:

Cadencia	Ejemplo de frecuencia	Propósito
Team Kanban Meeting	Diaria	Observar y seguir el estado y flujo del trabajo (no de los trabajadores). ¿Cómo podemos entregar los elementos de trabajo más rápido en el sistema? ¿Hay capacidad disponible? ¿Qué debemos tomar a continuación?
Team Retrospective	Quincenal o mensual	Reflexionar sobre cómo el equipo gestiona su trabajo y cómo pueden mejorar
Team Replenishment Meeting	Semanalmente o a demanda	Seleccionar los elementos de la lista de trabajo para realizar a continuación

Métricas de software - clase 23/10

En el enfoque tradicional:

triple restricción: alcance - costo - tiempo
La manera de planificación es a partir de un alcance que se acuerda, se derivan los tiempos y los costos, pero surgen cambios, que implica modificar el plan original, es parte del rol del líder del proyecto evaluar cómo tomar esos cambios y hacer variar esas variables de la triple restricción. Si no se puede realizar el cambio, hay que proponer otra solución. La calidad es innegociable. se tiene que negociar lo que aporte menos valor al negocio. En términos de requerimientos esos costos/funcionalidades no pueden cortarse, por eso se ajustan las otras variables.

Monitoreo de control - Métricas:
valor que se toma con respecto a la realidad, que nos permite evaluar la salud de proyecto, del proceso y del producto
Dominio de las métricas:
describen el ámbito en el cual esta métrica se aplica

- Métricas de proceso → misma métrica para el proyecto, pero en términos de un conjunto de proyectos (por ej. los del último año → cuánto cambió el costo)
  me interesa van los proyectos en mi organización - para saber si el método que estoy utilizando para los procesos está funcionando
- Métricas de proyecto → mientras dura la vida del proyecto → determina cómo vamos y cómo resultó el proyecto (tiempo, costos)
  me interesa mi proyecto
- Métricas de producto (bug, alcance, líneas de código)
  me interesa mi producto de software

Métricas básicas:

- Tamaño del producto
- Esfuerzo
- Tiempo (calendario)
- Defectos

Ejemplos:

Métricas del desarrollador:

1. esfuerzo
2. esfuerzo y duración estimada y actual de una tarea
3. % de cobertura por el unit test
4. número y tipo de defectos encontrados en el unit tests
5. número y tipo de defectos encontrados en revisión por pares

Métricas de la organización:

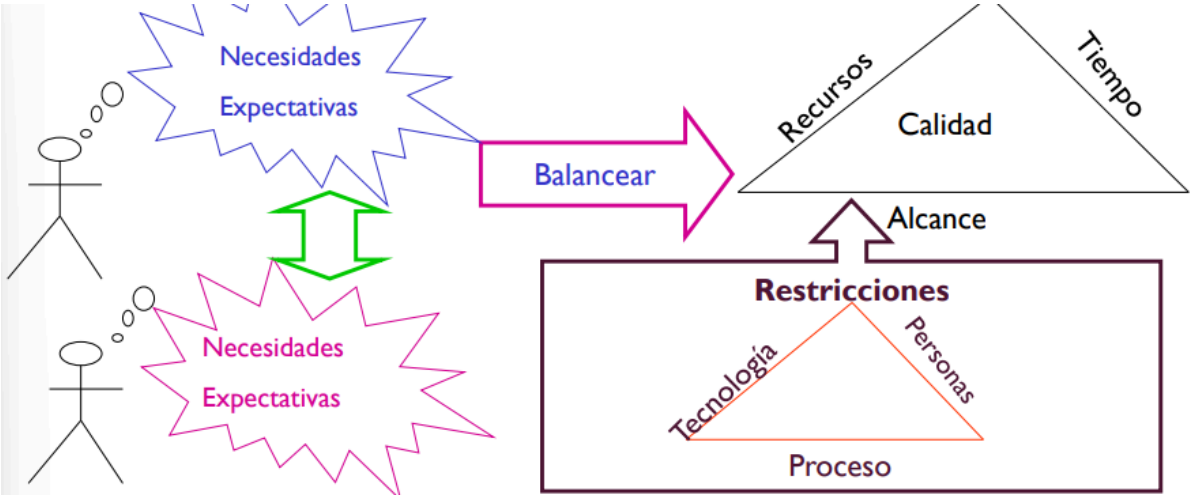
1. tiempo calendario

Métricas del equipo de desarrollo:

1.

La métrica no se usa para medir la performance de las personas. Tiene que responder a estas preguntas:

- Nos da más información que la que tenemos ahora?
- Es esta información de beneficio práctico?
- Nos dice lo que queremos saber?
- Cuando tengo que ver esa métrica? → cuando todavía estés a tiempo de corregir



balancear la triple restricción teniendo en cuenta las necesidades/expectativas/tecnologías

métrica: me dice que está pasando

estimación: predecir lo que va a pasar

Métricas en ambientes ágiles:

software funcionando la mejor medida de progreso → la métrica va a ir de la mano con esto
tratamos de eliminar el desperdicio → mido solo lo que es necesario

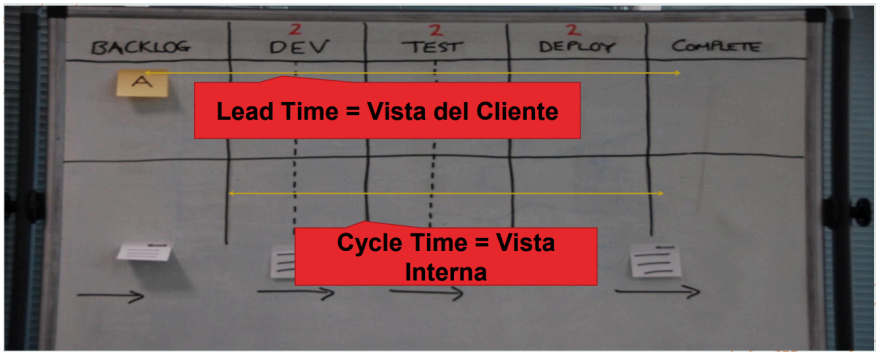
principios que guían las métricas:

- “Nuestra mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas de software valioso, funcionando."
- “El Software funcionando es la principal medida de progreso."

→ Velocidad : cantidad de story points de las user stories que el product owner nos confirmó en la review. Se espera que la velocidad sea constante a medida que van pasando los sprints. se ve el análisis en un sprint
→ Capacidad : para un sprint específico, es una métrica que se estima. su entrada es la velocidad, la capacidad surge de ahí, es la forma de estimar la capacidad del equipo. En los primeros sprints se hace en horas porque no se todavía la velocidad del equipo.
se calcula en el planning

→ Running tested features (RTF) : se analiza algo muy parecido a la velocidad pero a lo largo de todos los sprints
se calcula en la review

Métricas en Kanban



→ **Cycle Time (Tiempo de ciclo)**
Métrica que registra el tiempo que sucede entre el inicio y el final del proceso, para un ítem de trabajo dado. Se suele medir en días de trabajo o esfuerzo.
Medición más mecánica de la capacidad del proceso
- Ritmo de Terminación : tiempo que tardo en hacerla
→ **Lead Time (Tiempo de entrega)**
Métrica que registra el tiempo que sucede entre el momento en el cual se está pidiendo un ítem de trabajo y el momento de su entrega (el final del proceso). Se suele medir en días de trabajo.
- Ritmo de entrega
→ **Touch Time (Tiempo de Tocado)**
Tiempo en el cual un ítem de trabajo fue realmente trabajado (o “tocado”) por el equipo.

Cuantos dias hábiles paso este ítem en columnas de “trabajo en curso”, en oposición con columnas de cola/buffer y estado

$$Touch\ Time \leq Cycle\ Time \leq Lead\ Time$$

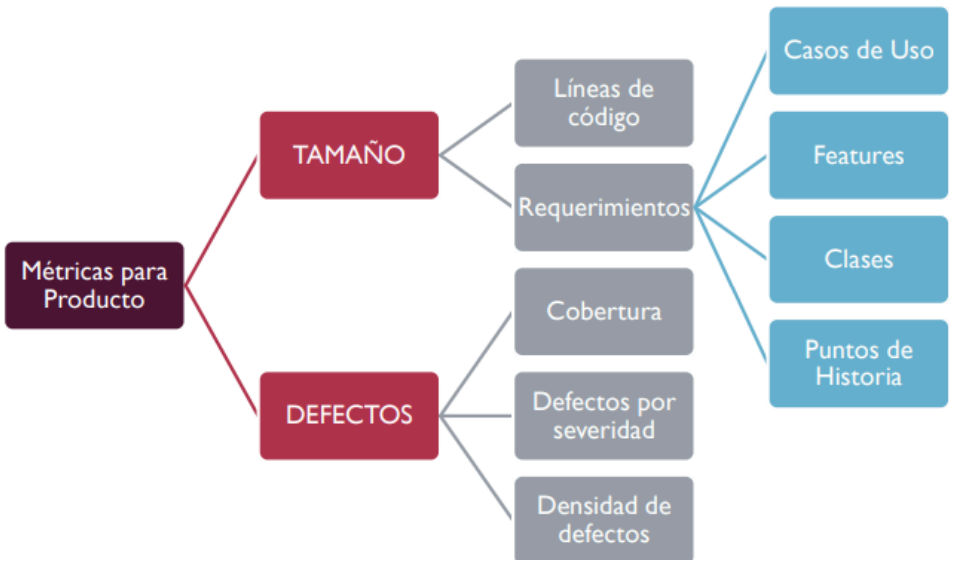
→ **Eficiencia del Ciclo de Proceso**
% Eficiencia ciclo proceso = Touch Time / Elapsed Time (Lead Time)

Métricas orientadas a servicio:
- expectativa de nivel de servicio que los clientes esperan
- capacidad del nivel de servicio al que el sistema puede entregar
- acuerdo de nivel de servicio que es acordado con el cliente
- umbral de la adecuación del servicio el nivel por debajo del cual este es inaceptable para el cliente

no mide incremento de producto - tiene que ver con el servicio que brindo
cumplir con mi lead time
cumplir con la expectativa de mi cliente

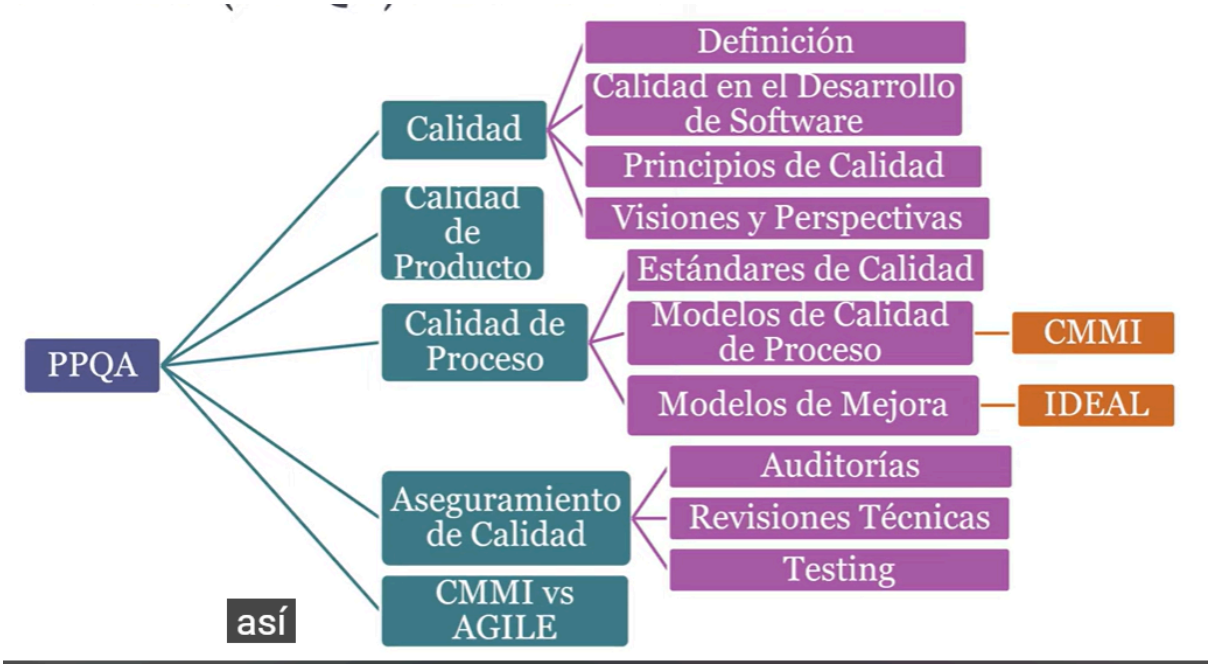
- **Tradicionales**
  - Esfuerzo
  - Tiempo
  - Costos
  - Defectos
- **Ágiles**
  - Velocidad
  - Capacidad
  - Running Tested Features
- **Lean**
  - Lead Time
  - Cycle Time
  - Touch Time
  - Eficiencia Proceso

Métricas del producto de software: (del enfoque tradicional)



Para los tres enfoques:
Medimos para entender, no para juzgar → para ver que mejorar

Notas UNIDAD 4

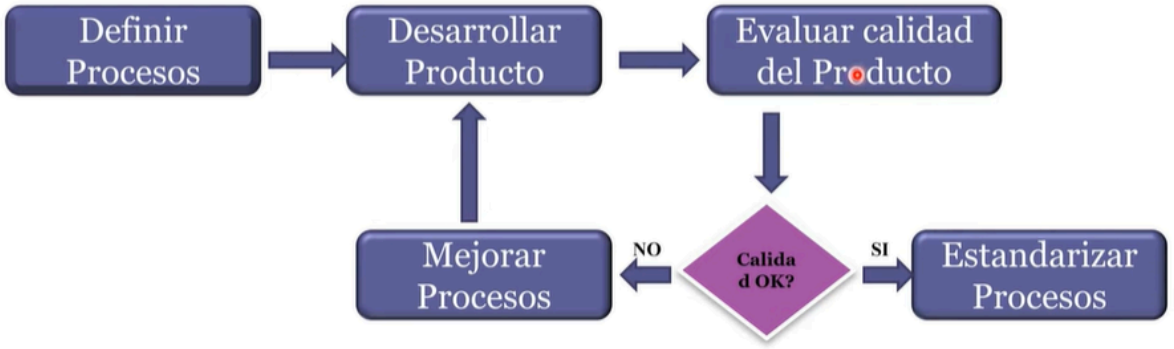




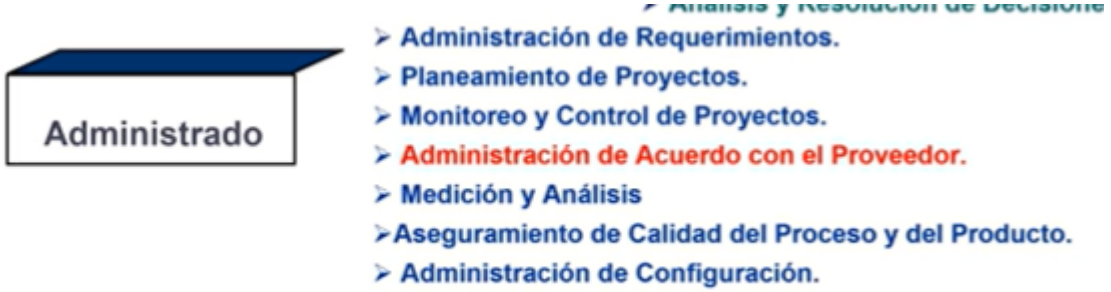
proyectos utilizan procesos para generar productos
procesos definidos o empíricos
modelos de mejora: plantillas para encarar un proceso ya definido para mejorarlo
modelos de calidad - de creación

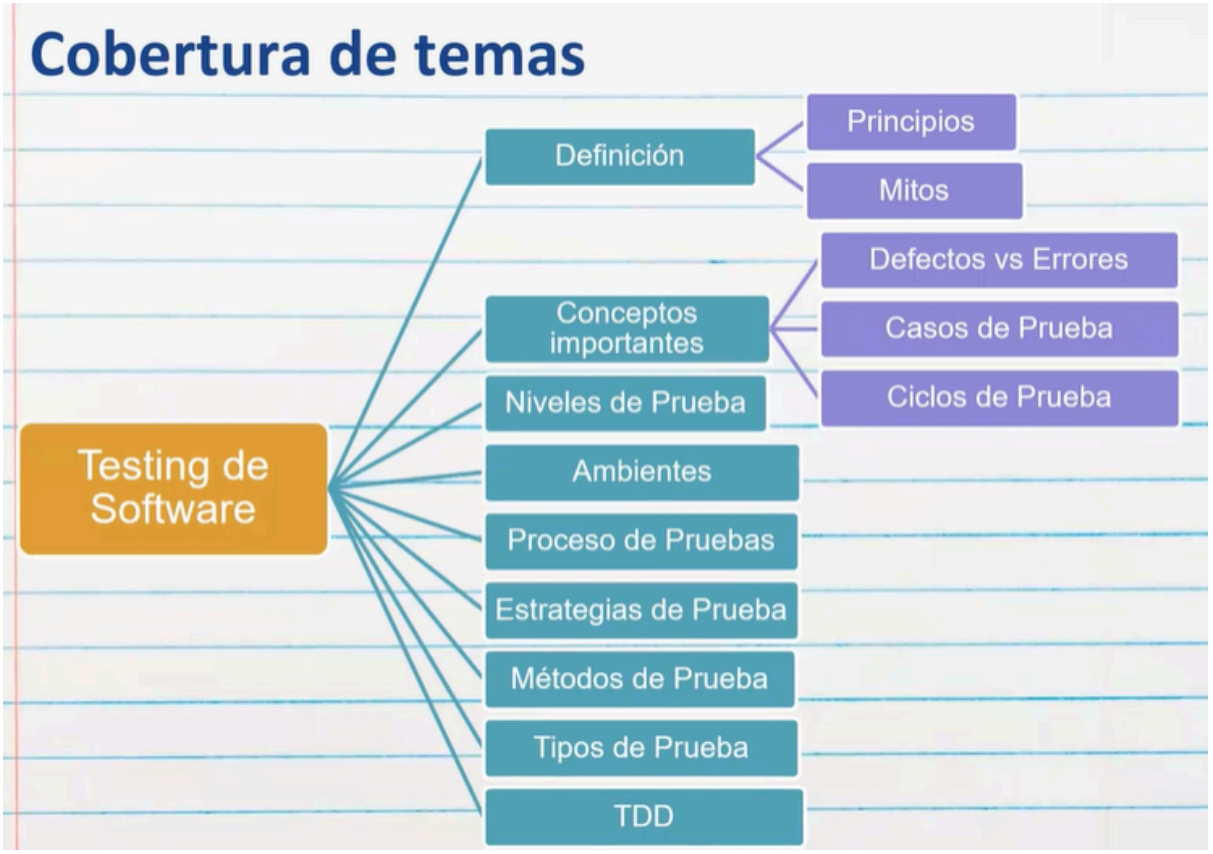
modelo para crearlos → CMMI descriptivos, plantea objetivos
modelo para mejorarlos → IDEAL prácticas, como cumplir esos objetivos
modelo para evaluarlos → xcamping (Revisiones, Auditorías)

Procesos basados en calidad

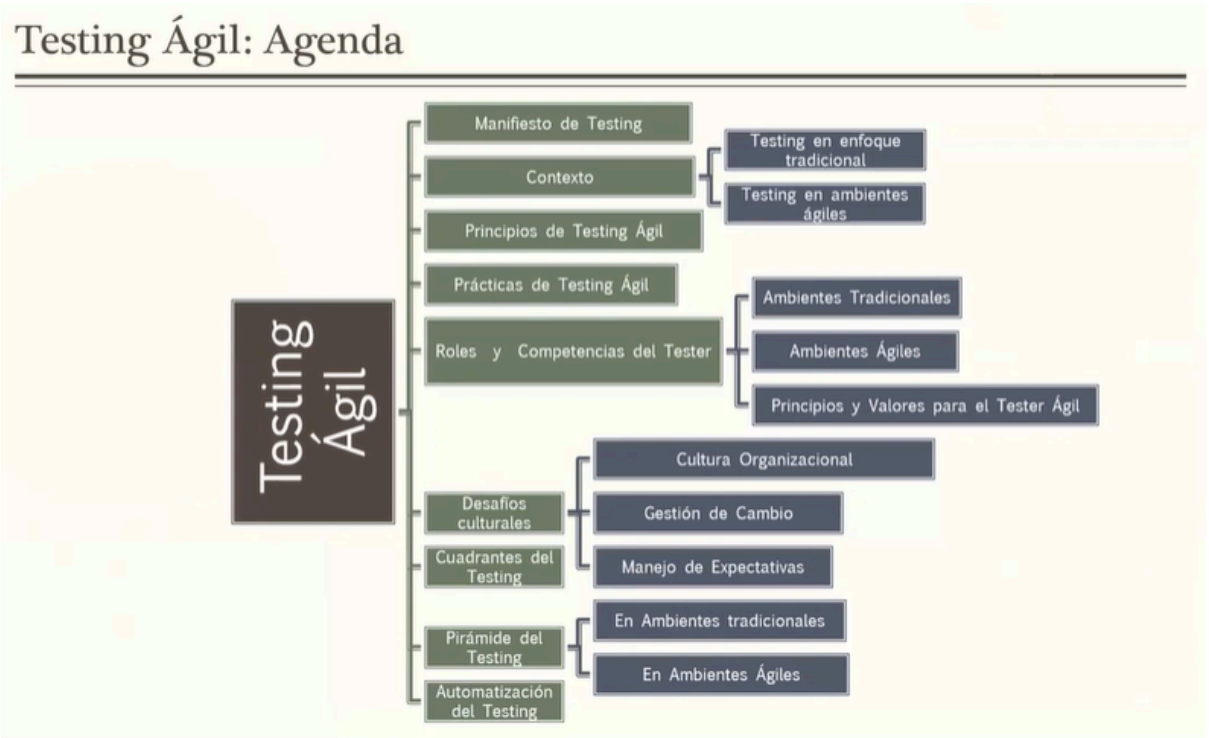


nivel 2:





Testing Ágil: Agenda



¿Cómo aplicar Kanban en nuestro proyecto?

- **Proceso**: modelar nuestro proceso.
- **Trabajo** : decidir la unidad de trabajo.
- **Límites de WIP**: limitar el WIP para ayudar al flujo de trabajo.
- **Política**: definir políticas de calidad.
- **Cuellos de Botella y Flujo**: mover recursos a los cuellos de botella.
- **Clase de Servicio**: diferentes trabajos tienen diferentes políticas – definición de hecho ("done"), para cada estado.
- **Cadencia**: Releases, planificaciones, revisiones